```
import UserNotifications

class NotificationRouter: NSObject, UNUserNotificationCenterDelegate {
    @MainActor
    func userNotificationCenter(
        _ center: UNUserNotificationCenter,
        didReceive response: UNNotificationResponse
    ) async {
»       let userInfo = response.notification.request.content.userInfo
»       if let path = userInfo["path"] as? String {
»           let url = baseURL.appending(path: path)
»       }
    }
}
```

But now that you have a URL, how do you get it back to a `Navigator`?

Enter `NavigationHandler`. Hotwire Native provides this protocol for exactly this reason—to get back to the Hotwire world from native code.

```
// hotwire-native-ios:Source/NavigationHandler.swift

import Foundation

/// A protocol to bridge back to Hotwire world from a native context. Use
/// this to trigger a new page visit including routing and presentation.
///
/// When responding to `NavigatorDelegate.handle(proposal:navigator:)`, to
/// route a native view controller, pass in an instance of `Navigator` typed
/// as this protocol with a weak reference. This ensures you avoid a
/// circular dependency between the two.
///
/// - Note: See `NumbersViewController` in the demo app for an example.
public protocol NavigationHandler: AnyObject {
    func route(_ url: URL)

    func route(_ proposal: VisitProposal)
}

extension Navigator: NavigationHandler {
    public func route(_ url: URL) {
        route(url, options: VisitOptions(action: .advance), parameters: nil)
```